Rotaract Club of Xiamen University Malaysia

Rotary Club of Kuala Lumpur Dynamics

Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: [25.10.2026]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	HamJinBengs
Project / Solution Name	Gantian
Name (Leader)	Tay Kyhenn
Email & Phone (Leader)	DSC2409061@xmu.edu.my, +6012-5083313
University Affiliation(s)	Xiamen University Malaysia (XMUM)
Live Prototype URL	- (haven't deployed to website yet)
Code Repository URL (public / access granted to judges)	https://github.com/Imonka/HamJinBengs

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

The system implements a layered, end-to-end architecture:

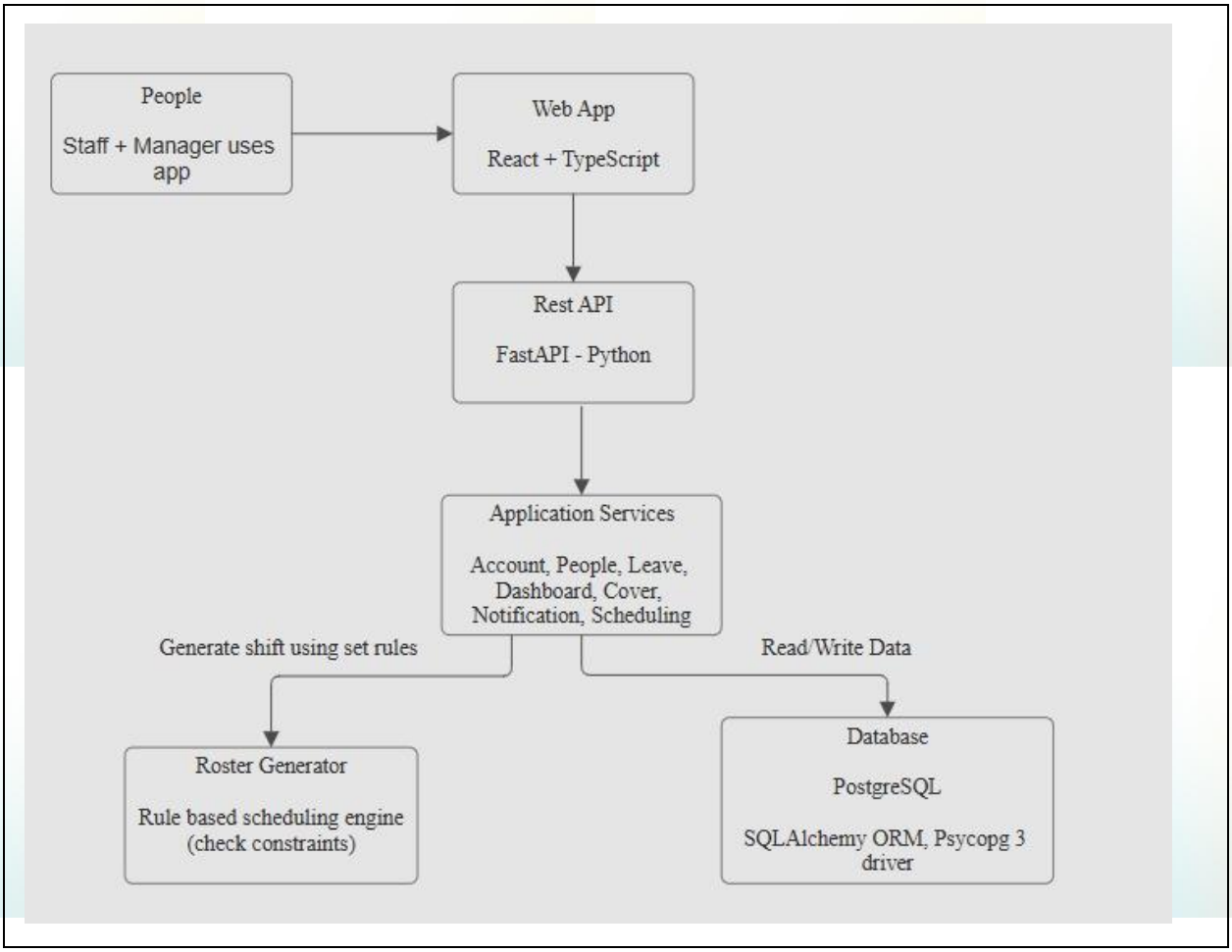
Frontend -> Backend API: Staff and Managers interact via a Web App built with React + TypeScript. All user actions trigger HTTP requests to the backend REST API layer developed with FastAPI - Python.

API -> Business Logic: The REST API serves as the gateway and routes requests to the Application Services layer, which encapsulates core domains: Account, People, Leave, Dashboard, Cover, Notification, and Scheduling.

Business Logic -> Engine & Database: The flow then splits. For rostering, the Scheduling service invokes the Roster Generator - a rule-based scheduling engine that generates shifts using set rules while checking constraints. For persistence, all services perform Read/Write Data operations to the database.

Database: Data is stored in PostgreSQL, accessed via SQLAlchemy ORM with Psycopg 3 driver. The response path reverses, returning JSON from the API to the Web App for rendering.

Appendix - System Block Diagram:


People
Staff + Manager uses app
Web App
React + TypeScript
Rest API
FastAPI - Python
Application Services
Account, People, Leave, Dashboard, Cover, Notification, Scheduling
Generate shift using set rules
Read/Write Data
Roster Generator
Rule based scheduling engine (check constraints)
Database
PostgreSQL
SQLAlchemy ORM, Psycopg 3 driver

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.13.11	Code – backend programming language	PSF
TypeScript 5.9.3	Code – frontend programming language	Apache – 2.0
React 19.1.0	Code – frontend UI framework	MIT
FastAPI 0.153.0	Code – backend API framework	MIT
PostgreSQL 16.10	Database – relational database system	PostgreSQL License

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
-	-	-

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
-	-	-

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
-	-	-	-

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
-	-	-	-	-

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Replit (Claude Opus 5.5)	Assisting in code writing, debugging, and running the project.	Major
Gemini (Argon)	Assisting in setting up local host for the system	Minor

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

We will cross-check the AI-generated code to ensure it aligns with our proposed system design. Upon completion, the system will be executed and subjected to repeated functional and logic testing. This ensures that the implementation accurately reflects our team's concept and that the system operates as intended.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Replit	It was used for assisting in code writing and system execution.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Yes. While the AI model itself could potentially be replaced, Replit's platform was highly essential to our workflow. Its integrated development environment and built-in design features were exceptionally convenient and efficient. This unified setup significantly accelerated our development process, enabling us to fully build and realize the core functionalities of our proposed system

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

Our application retrieves specific user data such as employees' name, position, work time, etc, directly from the Orbyte ecosystem database. We do not independently collect additional personal data.

To ensure privacy, the data is protected by using role-based access controls and secure API communication within our repository architecture. We retain this data only for as long as the user's account remains active in the Orbyte's ecosystem.

Additionally, no personal user data or PII (Personally Identifiable Information) is sent to external third-party services or generative AI APIs

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

Since our AI component relies on a rule-based system and a greedy search algorithm rather than a generative model, it avoids issues like hallucination or API rate limits. However, it presents distinct technical limitations.

First, the greedy search algorithm focuses on immediate optimal choices, which means it may converge on a local optimum rather than the global optimum, potentially yielding suboptimal final results. Second, the rule-based system is inherently rigid; it can struggle to process dynamic, unforeseen edge cases that fall outside our predefined logic constraints.

To mitigate these limitations, we designed the system with a 'human-in-the-loop' approach. The algorithm rapidly generates a highly optimized baseline, but provides administrators with the flexibility to manually override and fine-tune the final output. Furthermore, we designed our architecture to easily accommodate ongoing manual refinements to the rule engine.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

[Signature]

[Date]